

Linux Fundamentals: Hands-on Assignments on File I/O Operations

Assignment 0: Basic File and Directory Operations

Tasks

1. Create a main directory named **linux_practice_lab**.
2. Inside it, create three directories named **documents**, **notes**, and **backup**.
3. Move into the **documents** directory and create three empty files: **intro.txt**, **commands.txt**, and **practice.txt**.
4. Add at least five Linux commands you learned into **commands.txt**.
5. Display the content of **commands.txt** using **cat**, **head**, and **tail**.
6. Copy **commands.txt** into the **backup** directory.
7. Rename **practice.txt** to **daily_practice.txt**.
8. Move **intro.txt** from **documents** to **notes**.
9. Create a hidden file named **.secret_notes** inside the main directory and then list it using an option that shows hidden files.
10. Create a nested directory structure named **projects/week1/day1** using a single command.
11. Use both relative and absolute paths to list files from two different locations.
12. Remove one unnecessary test file that you create yourself, but do not remove any required assignment file.
13. Create a file named **lab_observation.txt** explaining what each of these commands does: **pwd**, **ls**, **cd**, **mkdir**, **touch**, **cp**, **mv**, **rm**, and **rmdir**.

Expert-Level Stretch Tasks

- Create the full directory structure again using fewer commands.
- Use command options to show file sizes in human-readable format.
- Find all **.txt** files inside **linux_practice_lab**.
- Compare the original **commands.txt** and the backup copy to confirm both contain the same content.
- Write a short note on when **rm** can be dangerous and how a learner should use it carefully.

Assignment 1: Command Stream Control and Evidence Collection

Scenario: You are working as a junior Linux administrator. Your trainer has asked you to collect system evidence from the terminal and save it into organized files without copying and pasting manually.

Learning Outcomes

- Understand the difference between standard input, standard output, and standard error.
- Use **>**, **>>**, **<**, **2>**, and **2>&1** correctly.
- Create evidence files using commands such as **whoami**, **id**, **pwd**, **ls**, **date**, **uname**, **wc**, and **cat**.
- Separate successful command output from error output.

Tasks

1. Create a directory named **file_io_assignment1** and inside it create three subdirectories: **reports**, **logs**, and **errors**.
2. Create a file named **reports/system_profile.txt** using output redirection. It must contain the current date, logged-in user, user ID details, current working directory, shell name, and kernel information.
3. Append the output of **ls -la** for your home directory into **reports/system_profile.txt** without overwriting earlier content.
4. Run one valid command and one intentionally invalid command. Redirect normal output to **logs/success.log** and error output to **errors/error.log**.
5. Run a command that lists both an existing path and a non-existing path. Save stdout and stderr in two separate files.
6. Repeat the previous task, but combine stdout and stderr into one file named **logs/combined_output.log**.
7. Create a text file named **input_names.txt** with at least ten names. Use input redirection with **sort** and save the sorted output to **reports/sorted_names.txt**.
8. Use **wc** to count lines, words, and characters in **input_names.txt** and append the result to **reports/system_profile.txt**.
9. Create a short reflection file named **reports/reflection.txt** explaining the difference between overwrite redirection and append redirection using your own words.

Assignment 2: Log Processing Pipeline and File I/O Automation

Scenario: You are given a raw application log file. Your task is to process it using Linux command-line tools and create a clean summary report using pipelines and File I/O operations.

Starter Data

Create a file named **app.log** with at least 30 lines. The file must contain a mix of log levels such as **INFO**, **WARNING**, **ERROR**, and **DEBUG**. Each line should include a date, time, log level, module name, and message.

Learning Outcomes

- Use pipes to connect multiple commands.
- Filter file content using **grep**, **sort**, **uniq**, **head**, **tail**, and **wc**.
- Use **tee** to display output and save it at the same time.
- Generate a meaningful report from raw text files.
- Redirect unwanted output to **/dev/null** where appropriate.

Tasks

1. Create a directory named **file_io_assignment2** with subdirectories **raw**, **processed**, **reports**, and **archive**.
2. Place **app.log** inside the **raw** directory.
3. Extract all **ERROR** lines from **raw/app.log** and save them to **processed/errors_only.log**.
4. Extract all **WARNING** lines and append them to **processed/warnings_only.log**.
5. Create **reports/log_summary.txt** containing the total number of log lines, number of ERROR lines, number of WARNING lines, and number of INFO lines.
6. Build a pipeline that identifies the top three most frequently occurring log levels and save the result to **reports/top_log_levels.txt**.
7. Use **tee** so that the top log levels appear on the screen and are also saved to the report file.
8. Search for one keyword such as **login**, **payment**, **network**, or **database** and save matching lines to **processed/keyword_matches.log**.
9. Run a command that may produce unnecessary errors and redirect those errors to **/dev/null**.
10. Create a final report named **reports/final_observation.txt** explaining what the log file indicates and which issue appears most frequently.

11. Archive the complete assignment directory into a compressed file named **file_io_assignment2.tar.gz**.